

 GAMERSTORE · DOCUMENTACIÓN TÉCNICA

Documentación · Frontend

React + Vite · qué hace cada archivo y qué modificar

Sistema de venta de tecnología · **Spring Boot** (API REST) + **React** (Vite) + **MySQL**

Los apartados marcados con  indican qué se puede modificar; los de , puntos delicados.

1. Cómo está organizado

- 1.1 Estructura de carpetas
- 1.2 Las dos zonas de la aplicación
- 1.3 Cómo arranca la app (main.jsx)
- 1.4 Cómo Vite habla con el backend (vite.config.js)

2. La capa de API (lo más importante de entender)

- 2.1 api/client.js — el fetch centralizado
- 2.2 api/endpoints.js — el catálogo de URLs
 - AuthAPI
 - PublicAPI (sin login)
 - AdminAPI (requiere JWT)
 - PagosAPI



Cómo agregar un endpoint nuevo

3. Estado global (los Context)

- 3.1 auth/AuthContext.jsx — la sesión
- 3.2 carrito/CarritoContext.jsx — el carrito
- 3.3 config/ConfigContext.jsx — la configuración de la tienda

4. Hooks propios

- 4.1 useTableControls(rows, opciones) — búsqueda, orden y paginación
- 4.2 useAutoClear(valor, limpiar, ms = 5000) — mensajes que se borran solos
- 4.3 useDuplicado(valor, verificar, opciones) — validación en vivo con debounce

5. Componentes reutilizables (components/ui)

- 5.1 El patrón provider: Toast y Confirm
- 5.2 Detalles útiles del resto

6. La tienda pública

- 6.1 Componentes de components/public
- 6.2 Las páginas
- 6.3 Carrito.jsx — control de stock en la interfaz
- 6.4 Checkout.jsx — el wizard de 4 pasos
 - Integración de Stripe (tokenización en el navegador)
 - El bloque de Yape
 - La animación de procesado
 - La confirmación



Qué se puede modificar en el Checkout

7. El panel admin

- 7.1 El patrón común de las páginas CRUD
- 7.2 Página por página
 - Login.jsx — el bloqueo con contador
 - Dashboard.jsx — KPIs y gráficos
 - AdminProductos.jsx — subida de imagen
 - AdminClientes.jsx — RENIEC

AdminPedidos.jsx — la página más completa

AdminVentas.jsx — registro de ventas con IGV

AdminPagos.jsx — la página fuera del menú

7.3 Los componentes de components/admin

8. Estilos (index.css)

8.1 Las variables de :root

8.2 Las familias de clases

9. Guía rápida: "quiero cambiar X, ¿dónde toco?"

Cómo ejecutarlo

Proyecto: GamerStore · **Stack:** React 18.3 + Vite 5.4 + React Router 6.26 + Recharts 2.12 + Stripe ([@stripe/react-stripe-js](#)) **Código:** [frontend/src](#) · **Backend:** Spring Boot en <http://localhost:8080>

Leyenda usada en todo el documento:

-  = **parte modificable:** puedes cambiarlo sin romper nada más.
-  = **parte delicada:** si lo tocas, revisa qué más depende de ello.

1. Cómo está organizado

1.1 Estructura de carpetas

Todo el frontend vive dentro de [frontend/](#) . El código fuente está en [frontend/src](#) , repartido en carpetas por **responsabilidad** (no por página):

Carpeta	Qué contiene	Cuándo tocarla
src/api/	client.js (fetch con JWT) y endpoints.js (todas las URLs del backend)	Cuando agregas/cambias un endpoint del backend
src/auth/	AuthContext.jsx (sesión) y ProtectedRoute.jsx (guardia de rutas)	Cuando cambias cómo se inicia/cierra sesión
src/carrito/	CarritoContext.jsx — el carrito de compras completo	Cuando cambias reglas del carrito (stock, persistencia)
src/config/	ConfigContext.jsx — config pública de la tienda (WhatsApp, Yape, Stripe)	Cuando agregas un dato de configuración global
src/hooks/	useTableControls , useAutoClear , useDuplicado	Cuando cambias comportamiento compartido (tiempos, paginación)
src/utils/	format.js — money() , waUrl() , sku()	Cuando cambias cómo se formatea un precio o un SKU
src/components/ui/	9 componentes genéricos: Modal, Toast, Confirm, tablas, skeletons	Cuando quieres que un patrón visual se vea igual en toda la app
src/components/public/	Navbar, Footer, layout y tarjetas de la tienda	Cuando cambias la cabecera/pie/tarjeta de producto
src/components/admin/	Sidebar, Topbar, layout, SessionTimer y PasarelaPago del panel	Cuando cambias el menú, la barra superior o el cobro
src/pages/public/	6 páginas de la tienda (Home, Catálogo, Detalle, Contacto, Carrito, Checkout)	Cuando cambias el contenido de una pantalla pública
src/pages/admin/	9 páginas del panel (Login, Dashboard y 7 pantallas de gestión)	Cuando cambias una pantalla del panel
src/index.css	Todo el CSS de la aplicación (751 líneas, un solo archivo)	Cuando cambias colores, espaciados o agregas una clase

Además, fuera de `src/` :

Archivo	Qué hace
<code>frontend/index.html</code>	HTML base: carga la fuente Inter y los iconos Bootstrap Icons desde CDN, y monta el <code><div id="root"></code>
<code>frontend/vite.config.js</code>	Puerto de desarrollo, proxy hacia el backend y carpeta de salida del build
<code>frontend/package.json</code>	Dependencias y los 3 scripts: <code>dev</code> , <code>build</code> , <code>preview</code>

1.2 Las dos zonas de la aplicación

La app es **una sola SPA** (Single Page Application) con dos zonas muy distintas, definidas en `src/App.jsx` :

Zona	Rutas	Layout	¿Necesita login?
Tienda pública	<code>/</code> , <code>/productos</code> , <code>/productos/:id</code> , <code>/contacto</code> , <code>/carrito</code> , <code>/checkout</code>	<code>PublicLayout</code> (Navbar + Footer)	No
Panel admin	<code>/admin</code> , <code>/admin/productos</code> , <code>/admin/categorias</code> , <code>/admin/clientes</code> , <code>/admin/pedidos</code> , <code>/admin/pagos</code> , <code>/admin/ventas</code> , <code>/admin/usuarios</code>	<code>AdminLayout</code> (Sidebar + Topbar)	Sí (<code>ProtectedRoute</code>)
Login	<code>/admin/login</code>	ninguno (pantalla propia)	No

Cualquier ruta que no exista redirige al inicio:

```
<Route path="*" element={<Navigate to="/" replace />} />
```

⚠ El anidado de rutas importa: `ProtectedRoute` envuelve a `AdminLayout` , que a su vez envuelve las páginas. Si sacas una página de ahí, deja de estar protegida.

1.3 Cómo arranca la app (`main.jsx`)

`main.jsx` monta `<App />` dentro de una **cadena de providers**. El orden no es casual: cada provider expone un contexto que los de adentro pueden consumir.

```
<BrowserRouter>      {/* habilita las rutas */}
  <ConfigProvider>     {/* config de la tienda */}
    <AuthProvider>     {/* sesión del admin */}
      <ToastProvider>   {/* notificaciones */}
        <ConfirmProvider>{/* diálogos de confirmación */}
          <CarritoProvider><App /></CarritoProvider>
```

⚠ Si mueves un provider hacia adentro de otro que lo necesita, obtendrás errores del tipo "no se puede leer de null" al llamar a `useAuth()` o `useToast()` .

1.4 Cómo Vite habla con el backend (vite.config.js)

En desarrollo corren **dos servidores a la vez**: Vite en el `:5173` y Spring Boot en el `:8080`. Para que el navegador no choque con CORS, Vite hace de **proxy**: recibe las peticiones a `/api` y `/images` y se las reenvía al backend.

```
server: {
  port: 5173,
  proxy: {
    '/api': 'http://localhost:8080',
    '/images': 'http://localhost:8080',
  },
},
build: {
  outDir: '../src/main/resources/static', // la SPA se compila DENTRO del proyecto Spring
  emptyOutDir: true,
}
```

Por eso en `client.js` la base es simplemente `const BASE = '/api'` (una ruta relativa): en desarrollo la resuelve el proxy y en producción la resuelve el propio Spring Boot, porque el build deja los archivos en `src/main/resources/static` y todo se sirve desde el mismo JAR.

-  Cambiar el puerto del front → `server.port`.
-  Cambiar el puerto del backend → las dos URLs del bloque `proxy`.
-  No cambies `outDir` sin avisar al backend: Spring espera la SPA exactamente en esa carpeta.

2. La capa de API (lo más importante de entender)

Toda comunicación con el backend pasa por **dos archivos**. Ninguna página hace `fetch` por su cuenta (salvo la subida de imágenes, que se explica más abajo). Esto es lo que hace que el proyecto sea mantenible: si cambia la autenticación, se cambia en **un** lugar.

2.1 `api/client.js` — el `fetch` centralizado

Este archivo resuelve cuatro problemas de una vez: guardar la sesión, mandar el token, renovarlo cuando vence y normalizar los errores.

a) La sesión vive en `localStorage` bajo tres claves:

Constante	Clave real	Qué guarda
<code>USER_KEY</code>	<code>gs_user</code>	El usuario (JSON: <code>username</code> , <code>nombre</code> , <code>rol</code>)
<code>TOKEN_KEY</code>	<code>gs_token</code>	El access token (JWT de vida corta)
<code>REFRESH_KEY</code>	<code>gs_refresh</code>	El refresh token (vida larga)

Funciones exportadas: `getUser()`, `getToken()`, `getRefreshToken()`, `saveSession({accessToken, refreshToken, user})`, `saveUser(user)` y `clearSession()`.

b) Agregar el token: `rawRequest()` arma el `fetch` y añade la cabecera solo si hay token:

```
const headers = {}
if (body !== undefined) headers['Content-Type'] = 'application/json'
if (token) headers['Authorization'] = 'Bearer ' + token
```

c) El **refresco silencioso (lo más interesante del archivo)**. El access token dura poco a propósito. Cuando vence, el backend responde **401**. En vez de expulsar al usuario, `request()` hace esto:

1. Lanza la petición con el token actual.
2. Si vuelve **401** (y no era la propia petición de login/refresh, y sí hay refresh token):
3. Llama a `refreshAccessToken()` → `POST /api/auth/refresh` con el refresh token.
4. Si el backend devuelve tokens nuevos, los guarda y **reintenta la misma petición**. El usuario no se entera de nada.
5. Si el refresh también falla → `clearSession()` + redirección a `/admin/login`.

```
if (res.status === 401 && !noRefresh && getRefreshToken()) {
  const nuevo = await refreshAccessToken()
  if (nuevo) res = await rawRequest(path, { method, body, token: nuevo })
  else { clearSession(); redirigirLogin(); throw new Error('Sesión expirada') }
}
```

⚠ **Single-flight:** si cinco peticiones reciben 401 al mismo tiempo, no se piden cinco tokens nuevos. La variable `refreshPromise` guarda la promesa en curso y todas comparten la misma; se libera en el `.finally()`. Si tocas esto,

ten cuidado de no quitar ese mecanismo.

d) Manejo de errores. Cuando la respuesta no es `ok`, se lanza un `Error` **enriquecido** con dos campos extra:

```
const err = new Error(msg)
err.status = res.status // código HTTP
err.data = data          // cuerpo JSON del backend
throw err
```

Esto permite que una pantalla reaccione a un caso concreto. El ejemplo real está en `Login.jsx`, que detecta el bloqueo por intentos fallidos:

```
if (err.status === 429 && err.data?.segundosRestantes) setBloqueo(err.data.segundosRestantes)
```

Además, si la respuesta es `204 No Content` (típico de un DELETE) devuelve `null`, y si no es JSON devuelve texto.

e) `downloadBlob(path, filename)` — descargas de PDF autenticadas. No se puede usar un `<a href>` normal porque el PDF requiere el header `Authorization`. Entonces: hace el fetch con token (con su propio reintento tras refresh), convierte la respuesta a `blob`, crea una URL temporal, simula un click en un `<a download>` y limpia todo con `URL.revokeObjectURL`.

Se usa en: boleta pública del checkout, reporte de pedidos, comprobantes de pago y boletas del panel.

f) El objeto `api` que consume `endpoints.js`:

```
export const api = {
  get, post, put, patch, del // todos pasan por request()
}
```

2.2 `api/endpoints.js` — el catálogo de URLs

Agrupamos **todas** las rutas del backend en cuatro objetos. Ninguna página escribe una URL a mano.

Incluye un helper `qs(params)` que convierte un objeto en query string ignorando valores vacíos/ `null` / `undefined` — por eso los filtros opcionales (fechas, estado) se pueden dejar en blanco sin ensuciar la URL.

AuthAPI

Función	Endpoint	Para qué
<code>login(username, password)</code>	<code>POST /auth/login</code>	Iniciar sesión, devuelve tokens + usuario
<code>me()</code>	<code>GET /auth/me</code>	Validar la sesión guardada al recargar la página
<code>logout()</code>	<code>POST /auth/logout</code>	Invaldar el refresh token en el servidor

PublicAPI (sin login)

Función	Endpoint	Para qué
<code>config()</code>	GET <code>/config</code>	Datos de la tienda: WhatsApp, Yape, clave pública de Stripe
<code>productos(categoria, q)</code>	GET <code>/productos?categoria=&q=</code>	Listado del catálogo con filtros
<code>producto(id)</code>	GET <code>/productos/:id</code>	Detalle + productos relacionados
<code>categorias()</code>	GET <code>/categorias</code>	Filtro lateral del catálogo
<code>reniec(dni)</code>	GET <code>/reniec/:dni</code>	Autocompletar nombres desde RENIEC en el checkout
<code>verificarCliente(data)</code>	POST <code>/checkout/cliente</code>	Identificar a un cliente ya registrado (DNI + email)
<code>checkout(data)</code>	POST <code>/checkout</code>	Cerrar la compra (cliente + items + pago)
<code>boletaUrl(pedidoCodigo, dni)</code>	<code>/checkout/boleta/:codigo?dni=</code>	Ruta de la boleta pública (se pasa a <code>downloadBlob</code>)

AdminAPI (requiere JWT)

Grupo	Funciones	Endpoints
Dashboar d	<code>dashboard()</code>	GET <code>/admin/dashboard</code>
Productos	<code>productos()</code> , <code>crearProducto()</code> , <code>actualizarProducto(id)</code> , <code>eliminarProducto(id)</code>	GET/POST/PUT/DELETE <code>/admin/productos</code>
Categoría s	<code>categorias()</code> , <code>crearCategoria()</code> , <code>actualizarCategoria(id)</code> , <code>eliminarCategoria(id)</code>	<code>.../admin/categorias</code>
Clientes	<code>clientes()</code> , <code>crearCliente()</code> , <code>actualizarCliente(id)</code> , <code>eliminarCliente(id)</code> , <code>buscarDni(dni)</code>	<code>.../admin/clientes</code> , GET <code>/admin/clientes/reniec/:dni</code>
Pedidos	<code>pedidos()</code> , <code>crearPedido()</code> , <code>actualizarPedido(id)</code> , <code>eliminarPedido(id)</code> , <code>reportePedidosUrl(params)</code>	<code>.../admin/pedidos</code> , <code>/admin/pedidos/reporte.pdf</code>
Comprob antes	<code>comprobantes(params)</code> , <code>resumenVentas(params)</code> , <code>boletaUrl(id)</code> , <code>boletaPedidoUrl(pedidoId)</code>	<code>/admin/comprobantes</code> , <code>/admin/comprobantes/resumen</code> , <code>.../:id/pdf</code> , <code>.../pedido/:id/pdf</code>
Usuarios	<code>usuarios()</code> , <code>crearUsuario()</code> , <code>actualizarUsuario(id)</code> , <code>eliminarUsuario(id)</code>	<code>.../admin/usuarios</code>
Uploads	<code>subirImagen(formData)</code>	POST <code>/api/admin/uploads</code>
Duplicado s	<code>existeProducto</code> , <code>existeCategoria</code> , <code>existeCliente</code> , <code>existeUsuario</code>	GET <code>.../existe?...</code>

⚠ `subirImagen()` es la **única excepción** que no usa `api.post` : como el cuerpo es un `FormData` (multipart), no puede llevar `Content-Type: application/json` , así que hace su propio `fetch` y arma el header `Authorization` leyendo directamente `localStorage.getItem('gs_token')` . Efecto secundario: **no tiene refresco automático**; si el token venció justo en ese momento, la subida falla y hay que reintentar.

PagosAPI

Función	Endpoint	Para qué
<code>listar()</code>	<code>GET /admin/pagos</code>	Historial de cobros
<code>config()</code>	<code>GET /admin/pagos/config</code>	Datos de la cuenta Yape + estado de Stripe
<code>pagarTarjeta(data)</code>	<code>POST /admin/pagos/tarjeta</code>	Cobrar un pedido con tarjeta
<code>pagarYape(data)</code>	<code>POST /admin/pagos/yape</code>	Cobrar con N° de operación de Yape
<code>comprobanteUrl(id)</code>	<code>/admin/pagos/:id/comprobante.pdf</code>	Ruta del comprobante (para <code>downloadBlob</code>)

✏ Cómo agregar un endpoint nuevo

1. Créalo en el backend (Spring).
2. **Regístralo aquí**, en el grupo que corresponda: `js // dentro de AdminAPI proveedores: () => api.get('/admin/proveedores'), crearProveedor: (data) => api.post('/admin/proveedores', data),`
3. Impórtalo y úsalo en la página: `AdminAPI.proveedores().then(setProveedores)` .

No toques `client.js` para esto: el token, el refresh y los errores ya vienen resueltos.

3. Estado global (los Context)

React Context evita el *prop drilling*: en vez de pasar el usuario o el carrito de componente en componente, se lee directamente donde haga falta con un hook.

3.1 `auth/AuthContext.jsx` — la sesión

Qué expone: `{ user, login, logout, isAuthenticated }` (`isAuthenticated` es simplemente `!!user`).

Cómo funciona:

- El estado arranca leyendo `localStorage` (`useState(() => getUser())`), así al recargar la página la sesión no se pierde.
- Un `useEffect` al montar **revalida** la sesión contra `AuthAPI.me()`. Si el access token venció pero el refresh sigue vivo, `client.js` lo renueva solo por debajo; si ambos fallaron, limpia la sesión.
- `login(username, password)` llama a la API, guarda `accessToken` + `refreshToken` + usuario y actualiza el estado.
- `logout()` avisa al backend para invalidar el refresh token, pero **siempre** limpia el navegador aunque esa llamada falle.

Cómo se usa: `const { user, logout } = useAuth()`. Lo consumen `Navbar`, `Sidebar`, `Topbar` y `Login`.

- ✏ Para mostrar más datos del usuario en la interfaz, amplía el objeto que se arma en `login()`: `const u = { username: r.username, nombre: r.nombre, rol: r.rol }`.
- ⚠ `ProtectedRoute` **no** usa `useAuth()`: llama a `getUser()` de `localStorage` directamente. Es más rápido (evita un parpadeo al recargar) pero significa que borrar la sesión solo del estado de React no bloquea el acceso.

3.2 `carrito/CarritoContext.jsx` — el carrito

Qué expone: `{ items, agregar, quitar, cambiarCantidad, limpiar, total, cantidadTotal }`.

Persistencia: se guarda en `localStorage` bajo `gs_carrito`. Un `useEffect` reescribe la clave cada vez que cambian los `items`, dentro de un `try/catch` (en modo incógnito `localStorage` puede fallar).

El tope de stock es la regla central del carrito, y está en una sola función:

```
const limitar = (cantidad, stock) => Math.max(1, Math.min(cantidad, stock))
```

Se aplica en los tres puntos donde puede cambiar la cantidad:

Acción	Comportamiento
<code>agregar(producto, cantidad)</code>	Si <code>stock <= 0</code> no hace nada. Si el producto ya estaba, suma las cantidades pero nunca pasa del stock
<code>cambiarCantidad(id, cantidad)</code>	Si la cantidad baja a <code>0</code> o menos, elimina el producto; si no, la limita al stock
<code>quitar(id)</code> / <code>limpiar()</code>	Quitan un producto o vacían el carrito

`total` y `cantidadTotal` se calculan con `reduce` en cada render — no se guardan en estado, así nunca quedan desincronizados.

Cómo se usa: `const { items, total, agregar } = useCarrito()` . Lo consumen `Navbar` (el badge), `ProductCard` , `ProductoDetalle` , `Carrito` y `Checkout` .

- ⚠ El `stock` se guarda **congelado** dentro del item cuando se agrega. Si en el panel alguien cambia el stock, el carrito del visitante seguirá con el valor viejo hasta que recargue el producto. La validación definitiva la hace el backend en `POST /checkout` .
- ✏ Cambiar la clave de persistencia → `const STORAGE_KEY = 'gs_carrito'` .

3.3 `config/ConfigContext.jsx` — la configuración de la tienda

Qué expone: el objeto de configuración completo que devuelve `GET /api/config` .

Define unos **valores de respaldo** para que la app funcione aunque el backend no responda:

```
const DEFAULTS = {
  tiendaNombre: 'GamerStore', whatsappNumero: '51986969024',
  yapeMontoMaximo: 500, stripeEnabled: false, stripePublicKey: '',
}
```

Al montar pide la config real y la reemplaza; si falla, se queda con los defaults (`.catch()` vacío a propósito).

Campos que consume el frontend:

Campo	Dónde se usa
<code>whatsappNumero</code>	<code>Footer.jsx</code> , <code>Contacto.jsx</code> (vía <code>waUrl()</code>)
<code>yapeMontoMaximo</code>	<code>Checkout.jsx</code> — bloquea Yape si el total lo supera
<code>yapeNumero</code> , <code>yapeTitular</code> , <code>yapeQr</code>	<code>Checkout.jsx</code> — bloque de pago con Yape
<code>stripeEnabled</code> , <code>stripePublicKey</code>	<code>Checkout.jsx</code> — decide entre Stripe real o formulario simulado

Cómo se usa: `const { whatsappNumero, yapeMontoMaximo } = useConfig()` .

- ⚠ `yapeNumero` , `yapeTitular` y `yapeQr` **no** están en `DEFAULTS` : llegan solo del backend. Si la API no responde, el checkout muestra el QR de respaldo ("Pide el QR al vendedor"), lo cual es intencional.
- ✏ Para agregar un dato global (por ejemplo un horario), agrégalo al endpoint `/api/config` del backend y a `DEFAULTS` aquí; queda disponible en toda la app sin más cambios.

4. Hooks propios

Tres hooks pequeños que eliminan repetición. Los tres viven en `src/hooks/`.

4.1 `useTableControls(rows, opciones)` — búsqueda, orden y paginación

⚠ **Concepto clave para la exposición:** este hook trabaja **enteramente en el navegador**. Recibe el arreglo completo que ya devolvió el backend y filtra, ordena y pagina en memoria. **No hace ninguna petición HTTP**. Por eso la búsqueda es instantánea, y por eso también funcionaría mal con decenas de miles de registros (habría que mover la lógica al backend).

Opciones: `{ searchKeys = [], pageSize = 8, initialSort = null }`.

Devuelve: `{ paged, query, onSearch, sort, toggleSort, page, setPage, totalPages, total }`.

Trabaja en tres pasos encadenados con `useMemo` (para no recalcular en cada render):

1. **Filtrar** — busca el texto (minúsculas, sin espacios extra) en las columnas indicadas en `searchKeys`.
2. **Ordenar** — si son números los compara numéricamente; si no, usa `localeCompare(..., 'es', { numeric: true })`, que ordena bien las tildes y los números dentro de textos.
3. **Paginar** — corta el arreglo con `slice()`.

Tanto `toggleSort()` como `onSearch()` hacen `setPage(1)`, porque al cambiar el orden o el filtro la página actual deja de tener sentido.

```
const t = useTableControls(productos || [], {
  searchKeys: ['nombre', 'categoriaNombre'],
  pageSize: 8,
  initialSort: { key: 'nombre', dir: 'asc' },
})
```

- ✏ Filas por página → el `pageSize` que pasa cada página (todas usan **8**), o el default del hook.
- ✏ Qué columnas busca el buscador → `searchKeys`.

4.2 `useAutoClear(valor, limpiar, ms = 5000)` — mensajes que se borran solos

Evita que un mensaje de error quede clavado en pantalla después de leerlo. Si `valor` tiene contenido, programa un `setTimeout` que llama a `limpiar('')`; el `return` del efecto cancela el temporizador si el valor cambia antes.

```
useAutoClear(formError, setFormError) // se borra a los 5 s
```

Lo usan `AdminProductos`, `AdminCategorias`, `AdminClientes`, `AdminPedidos`, `AdminUsuarios`, `Login` y `Checkout`.

- ✏ **Cambiar los 5 segundos:** el default está en la firma (`ms = 5000`). Para cambiarlo en todas partes edita ese número; para una sola pantalla pásalo como tercer argumento: `useAutoClear(formError, setFormError, 8000)`.

4.3 `useDuplicado(valor, verificar, opciones)` — validación en vivo con debounce

Avisa **mientras el usuario escribe** si un nombre/DNI/email ya existe, sin necesidad de enviar el formulario.

Opciones: `{ delay = 500, minLargo = 3, activo = true }`. **Devuelve:** `{ duplicado, mensaje, verificando }`.

Cómo funciona el *debounce*: cada tecla reinicia un temporizador de 500 ms; solo cuando el usuario **deja de escribir** se llama al backend. Sin esto se dispararía una petición por letra. Además usa un flag `cancelado` en la limpieza del efecto para descartar respuestas de peticiones viejas que llegan tarde (condición de carrera).

```
const dupNombre = useDuplicado(  
  form.nombre,  
  (v) => AdminAPI.existeProducto(v, editing?.id),  
  { activo: showModal }  
)
```

El resultado se usa para dos cosas: pintar el input de rojo (`input-error` + `<small class="campo-error">`) y **deshabilitar el botón Guardar** (`disabled={saving || dupNombre.duplicado}`).

-  **Cambiar el debounce (500 ms)** → default `delay` en la firma del hook, o `{ delay: 800 }` en una llamada puntual.
 -  **Cambiar el largo mínimo** → `minLargo`. Ejemplos reales del código: DNI usa `minLargo: 8`, email usa `5`, usuario usa `3`.
 -  `activo: showModal` es importante: sin eso el hook seguiría consultando al backend con el modal cerrado.
-

5. Componentes reutilizables (components/ui)

Nueve componentes sin dependencias externas. La regla del proyecto: si un patrón visual aparece en dos pantallas, se convierte en componente.

Componente	Para qué sirve	Props principales	Ejemplo de uso
<code>Toast.jsx</code>	Avisos flotantes arriba a la derecha, se cierran solos	(provider) — se usa vía <code>useToast()</code>	<code>toast.success('Guardado')</code>
<code>Confirm.jsx</code>	Diálogo "¿seguro?" que reemplaza a <code>window.confirm</code>	(provider) — se usa vía <code>useConfirm()</code>	<code>const ok = await confirm({ message: '¿Eliminar?', danger: true })</code>
<code>Modal.jsx</code>	Ventana modal genérica	<code>title</code> , <code>icon</code> , <code>onClose</code> , <code>footer</code> , <code>size</code> (<code>sm</code> / <code>lg</code>)	<code><Modal title="Nuevo" onClose={cerrar}>...</Modal></code>
<code>Alert.jsx</code>	Mensaje fijo dentro de la página o modal	<code>type</code> (<code>success</code> / <code>error</code> / <code>info</code>)	<code><Alert type="error">{formError}</Alert></code>
<code>Pagination.jsx</code>	Controles < 1 2 3 > + contador de registros	<code>page</code> , <code>totalPages</code> , <code>total</code> , <code>onPage</code>	<code><Pagination page={t.page} totalPages={t.totalPages} total={t.total} onPage={t.setPage} /></code>
<code>TableToolbar.jsx</code>	Barra superior de tabla: buscador + contador	<code>query</code> , <code>onSearch</code> , <code>total</code> , <code>right</code>	<code><TableToolbar query={t.query} onSearch={t.onSearch} total={t.total} /></code>
<code>TableSkeleton.jsx</code>	Esqueleto de carga con forma de tabla	<code>rows</code> (por defecto 6)	<code><TableSkeleton rows={4} /></code>
<code>Skeleton.jsx</code>	Rectángulo gris con efecto <i>shimmer</i>	<code>className</code> , <code>style</code>	<code><Skeleton className="sk-card" /></code>
<code>Spinner.jsx</code>	Círculo giratorio (animado por CSS)	ninguna	<code><Spinner /></code>

5.1 El patrón provider: `Toast` y `Confirm`

Estos dos no se usan como etiquetas sueltas. Se montan **una sola vez** en `main.jsx` y se consumen desde cualquier profundidad con un hook. Es el mismo patrón que usan librerías como `react-hot-toast`, pero escrito a mano.

Toast guarda una lista de avisos en estado. `mostrar()` agrega uno con un id incremental y programa su borrado:

```
const id = ++contador
setToasts((lista) => [...lista, { id, mensaje, tipo }])
setTimeout(() => quitar(id), 3500)
```

Expone tres métodos: `toast.success(m)`, `toast.error(m)`, `toast.info(m)`. El provider renderiza `{children}` y el contenedor `.toaster`, por eso los avisos aparecen sobre cualquier pantalla.

- ✏ **Duración del toast:** los `3500` ms de `Toast.jsx` (línea del `setTimeout`).

-  Los iconos están en el objeto `iconos` (`bi-check-circle-fill` , etc.).

Confirm es más ingenioso: convierte un diálogo visual en una **promesa**. `confirm()` devuelve una `Promise` y guarda su `resolver` en estado; cuando el usuario pulsa un botón, `cerrar(true/false)` resuelve esa promesa. Por eso se puede escribir código secuencial y legible:

```
const ok = await confirm({ title: 'Eliminar producto', message: '¿Seguro?', confirmText: 'Eliminar', danger: true })
if (!ok) return
```

Internamente reutiliza `Modal` con `size="sm"` . Con `danger: true` el icono pasa a advertencia y el botón a `btn-danger` .

5.2 Detalles útiles del resto

- `Modal` se cierra de tres formas: la X, click en el fondo (el `.modal-overlay`) y la tecla **Escape**. Además bloquea el scroll del body mientras está abierto (`document.body.style.overflow = 'hidden'`) y lo restaura al desmontarse. El click dentro del modal usa `stopPropagation()` para no cerrarse solo.
- `Pagination` devuelve `null` si `total === 0` (no se ve nada si no hay datos) y **pinta todos los números de página**. ⚠ Con muchas páginas la barra se alarga; si el proyecto crece, aquí habría que agregar puntos suspensivos.
- `TableToolbar` acepta una prop `right` para reemplazar el contador por lo que quieras (botones, filtros).

6. La tienda pública

6.1 Componentes de `components/public`

Componente	Qué hace
<code>PublicLayout.jsx</code>	Navbar + <code><Outlet /></code> (la página actual) + Footer
<code>Navbar.jsx</code>	Logo, buscador, enlaces, badge del carrito y acceso admin
<code>Footer.jsx</code>	4 columnas: marca/redes, navegación, categorías y contacto
<code>ProductCard.jsx</code>	Tarjeta de producto del catálogo
<code>ProductGridSkeleton.jsx</code>	Grilla de tarjetas fantasma mientras carga (<code>count</code> , <code>cols</code>)

`Navbar` tiene tres detalles a explicar:

- El buscador **navega**, no filtra: `navigate('/productos?q=' + encodeURIComponent(q))`. El estado de búsqueda vive en la URL.
- El badge solo aparece si hay algo: `{cantidadTotal > 0 && <span className="cart-badge">{cantidadTotal}</span>}`.
- Si hay sesión iniciada muestra "Panel" + "Salir"; si no, un botón "Admin".  El nombre y el icono de la marca están en `.nav-brand` (`<i className="bi bi-controller" /> GamerStore`).

`ProductCard` resuelve un conflicto típico: **toda la tarjeta es un `<Link>`**, pero dentro hay un botón "Agregar". Si no se frena la propagación, al agregar también navegarías al detalle:

```
const handleAgregar = (e) => {
  e.preventDefault()
  e.stopPropagation()
  agregar(p, 1)
  toast.success('Agregado al carrito')
}
```

6.2 Las páginas

Página	Ruta	Qué muestra	Endpoints
Home.jsx	/	Hero, 3 razones para comprar, 8 destacados , CTA final	PublicAPI.productos() (corta con .slice(0, 8))
Catalogo.jsx	/productos	Filtro lateral (búsqueda + categorías) y grilla	PublicAPI.categorias() , PublicAPI.productos(categoria, q)
ProductoDetalle.jsx	/productos/:id	Imagen, precio, stock, cantidad, especificaciones y relacionados	PublicAPI.producto(id)
Contacto.jsx	/contacto	WhatsApp, datos de la tienda y "¿cómo comprar?"	ninguno (usa useConfig())
Carrito.jsx	/carrito	Items, cantidades y total	ninguno (todo desde CarritoContext)
Checkout.jsx	/checkout	Wizard de 4 pasos + pago	verificarCliente , reniec , checkout , boletoUrl

Patrón de carga compartido: el estado arranca en `null` (no en `[]`) para poder distinguir "cargando" de "vacío":

```
const [productos, setProductos] = useState(null)
// ...
{productos === null ? <ProductGridSkeleton /> : productos.length === 0 ? <div className="empty">...</div> :
<grilla/>}
```

Catalogo guarda los filtros en la **URL** con `useSearchParams()` , no en estado local. Ventaja: el enlace se puede compartir y el botón "atrás" del navegador funciona. Un `useEffect` recarga los productos cada vez que cambia `categoria` o `q` .

ProductoDetalle carga producto + relacionados en una sola llamada (`const { producto: p, relacionados } = data`), hace `window.scrollTo(0, 0)` al cambiar de producto y limita la cantidad al stock directamente en el input:

```
onChange={(e) => setCantidad(Math.max(1, Math.min(Number(e.target.value) || 1, p.stock)))}
```

Tiene dos acciones: "Agregar al carrito" (se queda) y "Comprar ahora" (agrega y navega a `/checkout`).

Contacto es la página más fácil de editar: los datos y los pasos son dos arreglos al inicio del archivo (`info` y `pasos`). ✎ Editas el arreglo y la página cambia sola.

6.3 Carrito.jsx — control de stock en la interfaz

La página consume el contexto (`items, cambiarCantidad, quitar, total`) y no tiene lógica propia de negocio. Muestra un estado vacío con enlace al catálogo si `items.length === 0` .

Lo interesante es cómo se refleja el tope de stock en los botones:

```

<button onClick={() => cambiarCantidad(i.id, i.cantidad - 1)}>-</button>
<span>{i.cantidad}</span>
<button disabled={i.cantidad >= i.stock} onClick={() => cambiarCantidad(i.id, i.cantidad + 1)}>+</button>

```

- El botón + se deshabilita al llegar al stock: el usuario ve por qué no puede seguir.
- El botón - nunca se deshabilita: bajar de 1 llama a `cambiarCantidad(id, 0)` y el contexto interpreta eso como "quitar el producto".

Es un buen ejemplo de **dobles red de seguridad**: la interfaz previene (botón gris), el contexto corrige (`limitar()`) y el backend valida de verdad al hacer checkout.

6.4 Checkout.jsx — el wizard de 4 pasos

Es la pantalla más grande del proyecto (~990 líneas) y la que más conviene entender para exponer.

Los pasos se controlan con un solo número, `const [step, setStep] = useState(1)`, y sus nombres están en una constante:

```
const PASOS_LABEL = ['Identificación', 'Datos', 'Pago', 'Confirmar']
```

El *stepper* de arriba se genera recorriendo ese arreglo: un paso es `done` si `n < step`, `active` si `n === step`. El **resumen del pedido** (columna derecha) se ve en los 4 pasos.

Paso	Qué pide	Estado clave	Cómo se avanza
1 - Identificación	Elegir "invitado" o "ya soy cliente"	<code>modo</code> , <code>identDni</code> , <code>identEmail</code> , <code>identificado</code>	Invitado: botón directo. Cliente: <code>PublicAPI.verificarCliente()</code> y si acierta salta al paso 2 con los datos cargados
2 - Datos	DNI, nombres, apellidos, teléfono, email, dirección	<code>cliente</code>	<code>datosValidos</code> exige DNI de 8 dígitos + nombres + apellidos. Botón "Buscar" consulta RENIEC (<code>PublicAPI.reniec</code>) y autocompleta
3 - Pago	Yape o tarjeta	<code>metodo</code> , <code>numeroOperacion</code> , <code>tarjeta</code> , <code>paymentMethodId</code>	Con Stripe: <code>continuarConStripe()</code> . Sin Stripe: valida <code>pagoValido</code>
4 - Confirmar	Revisar todo y pagar	<code>procesando</code> , <code>pasoAnim</code> , <code>confirmacion</code>	<code>pagar()</code> → <code>POST /api/checkout</code>

Si la identificación como cliente falla, hay una **salida de emergencia**: el error trae un botón "Continuar como invitado" (`continuarComoInvitado()`) para no bloquear la venta.

Integración de Stripe (tokenización en el navegador)

Este es el punto técnicamente más fino. La idea: **el número de tarjeta nunca llega a nuestro servidor**.

1. La promesa de Stripe se crea **una sola vez** con `useMemo`, y solo si el backend activó la pasarela: `js const stripePromise = useMemo( () => (stripeEnabled && stripePublicKey ? loadStripe(stripePublicKey) : null), [stripeEnabled, stripePublicKey] )`

2. El formulario vive dentro de `<Elements stripe={stripePromise}>`, porque los hooks `useStripe()` y `useElements()` **exigen** ese contexto. Por eso existe el subcomponente `FormularioTarjetaStripe`.
3. `CardElement` es un iframe de Stripe: los datos de la tarjeta están fuera de nuestro DOM.
4. ⚠️ **La tokenización ocurre al salir del paso 3, no al pagar.** Motivo: en el paso 4 el formulario ya no se renderiza y el `CardElement` desaparecería. `continuarConStripe()` llama a `stripe.createPaymentMethod()`, guarda el `paymentMethodId` y un resumen (`{ marca, ult4 }`) para mostrarlo en la revisión.
5. Al pagar, el cuerpo cambia según el modo: `js pago: stripeEnabled && paymentMethodId ? { metodo: 'TARJETA', paymentMethodId } // solo el token : { metodo: 'TARJETA', numero, titular, vencimiento, cvv, ... } // modo simulado`
6. ⚠️ Si el pago se rechaza, el `paymentMethodId` **se descarta** (`setPaymentMethodId(null)`) y se vuelve al paso 3: un `PaymentMethod` ya usado no se puede reutilizar.

Modo simulado (Stripe desactivado): se usa un formulario propio con validación real en el navegador — algoritmo de Luhn (`luhn()`), formateo en grupos de 4 (`formatearTarjeta()`), MM/AA automático (`formatearVencimiento()`) y detección de marca por BIN (`detectarMarca()`): Visa empieza en 4, Mastercard 51-55 o 2221-2720, Amex 34/37, Discover 6011/65).

El bloque de Yape

- Muestra el **QR** (`yapeQr`) con respaldo si la imagen falla (`onError={() => setQrError(true)}`).
- Muestra número y titular, con botón **Copiar** (`navigator.clipboard.writeText`) que da feedback 1.5 s.
- Pide el **N° de operación** (solo dígitos, máx. 20; válido con ≥ 6).
- ⚠️ **Tope de S/ 500:** `const yapeBloqueado = total > yapeMontoMaximo`. Si se supera, la opción se deshabilita y un `useEffect` cambia el método a tarjeta automáticamente, para que nunca quede seleccionado un método imposible.
- Lleva un aviso ámbar: *"Pago de prueba: este QR es real (cuenta de {yapeTitular}). Si yapeas por error, el monto será devuelto."*

La animación de procesado

No es puramente decorativa: corre **en paralelo** a la llamada real y `Promise.all` espera a que ambas terminen.

```
const [resp] = await Promise.all([PublicAPI.checkout(body), animar()])
```

`animar()` avanza `pasoAnim` cada 600 ms por tres etapas. La del medio cambia de texto según el método:

```
const pasosAnimacion = ['Validando datos',  
  metodo === 'yape' ? 'Verificando la operación' : 'Autorizando con el banco',  
  'Confirmando tu pedido']
```

Esto garantiza un mínimo de ~1.8 s de espera: si el backend responde en 100 ms, el salto instantáneo se sentiría falso.

La confirmación

Si `confirmacion` tiene valor, la página entera se reemplaza por la pantalla de éxito (un `return` temprano, antes incluso de comprobar si el carrito está vacío — que lo está, porque `limpiar()` se ejecutó a propósito). Muestra código de pedido, código de pago, método, referencia, total, cliente y boleta.

Si el backend emitió comprobante, aparece **Descargar boleta**, que usa `downloadBlob` con la URL pública verificada por DNI:

```
await downloadBlob(PublicAPI.boletaUrl(confirmacion.pedidoCodigo, cliente.dni),
    'boleta-' + confirmacion.comprobanteCodigo + '.pdf')
```

 Qué se puede modificar en el Checkout

Quiero...	Dónde
Cambiar los nombres de los pasos	<code>const PASOS_LABEL = [...]</code> (arriba del archivo)
Agregar o quitar un paso	El arreglo <code>PASOS_LABEL</code> + un bloque <code>{step === N && (...)}</code> + los <code>setStep()</code> de los botones 
Cambiar los textos de la animación	<code>const pasosAnimacion = [...]</code>
Cambiar la velocidad de la animación	Los tres <code>esperar(600)</code> dentro de <code>animar()</code>
Cambiar el aviso de demo	Los bloques <code>.aviso-demo</code> (uno para Stripe, otro para el modo simulado, otro para Yape)
Cambiar el tope de Yape	Backend (<code>yapeMontoMaximo</code> de <code>/api/config</code>); el fallback local está en <code>DEFAULTS</code> de <code>ConfigContext</code>
Cambiar la validación de datos	<code>datosValidos</code> y <code>pagoValido</code>

7. El panel admin

7.1 El patrón común de las páginas CRUD

Cinco páginas comparten exactamente la misma estructura: `AdminProductos`, `AdminCategorias`, `AdminClientes`, `AdminUsuarios` y (con más piezas) `AdminPedidos`. Si entiendes una, entiendes todas.

1. Estado:

```
const [productos, setProductos] = useState(null) // null = cargando, [] = vacío
const [showModal, setShowModal] = useState(false) // ¿modal abierto?
const [editing, setEditing] = useState(null) // null = crear, objeto = editar
const [form, setForm] = useState(EMPTY) // campos del formulario
const [saving, setSaving] = useState(false) // bloquea el botón Guardar
const [formError, setFormError] = useState('') // error dentro del modal
```

2. `cargar()` — una línea que trae los datos y los deja en estado, con `.catch()` a arreglo vacío para que un fallo no deje la pantalla en "cargando" para siempre:

```
const cargar = () => AdminAPI.productos().then(setProductos).catch(() => setProductos([]))
useEffect(() => { cargar() }, [])
```

3. El modal, en dos modos. Un solo `<Modal>` sirve para crear y editar; la diferencia es `editing`:

- `abrirCrear()` → `setEditing(null)` + `setForm(EMPTY)`.
- `abrirEditar(p)` → `setEditing(p)` + `setForm({...datos de p})`.

Y el título se decide con un ternario: `title={editing ? 'Editar producto' : 'Nuevo producto'}`.

4. `guardar(e)` — decide POST o PUT según `editing`, muestra un toast, cierra el modal y **recarga la tabla**:

```
if (editing) { await AdminAPI.actualizarProducto(editing.id, payload); toast.success('Producto actualizado')
}
else { await AdminAPI.crearProducto(payload); toast.success('Producto creado correctamente') }
setShowModal(false)
cargar()
```

5. `eliminar(x)` — siempre pide confirmación primero:

```
const ok = await confirm({ title: 'Eliminar producto', message: `¿Seguro...?`, confirmText: 'Eliminar',
danger: true })
if (!ok) return
await AdminAPI.eliminarProducto(p.id); toast.success('Producto eliminado'); cargar()
```

6. La tabla usa `useTableControls` + los tres componentes de UI, y cada página define localmente un pequeño componente `Th` para las cabeceras ordenables (muestra ▲ ▼ ↕ según el orden actual):

```

<TableToolbar query={t.query} onSearch={t.onSearch} total={t.total} />
{ /* filas: t.paged.map(...) */ }
<Pagination page={t.page} totalPages={t.totalPages} total={t.total} onPage={t.setPage} />

```

7. Retroalimentación: `toast` para acciones puntuales, `Alert` + `useAutoClear` para errores dentro del modal, `useDuplicado` para el aviso en vivo.

Una convención que se repite: `<button type="submit" hidden />` dentro del `<form>`. Sirve para que **Enter** envíe el formulario, aunque el botón visible de Guardar esté en el `footer` del modal (fuera del `<form>`).

7.2 Página por página

Página	Ruta	Columnas de la tabla	Endpoints	Lo particular
Login.jsx	/admin/login	—	AuthAPI.login (vía useAuth)	Bloqueo con contador
Dashboard.jsx	/admin	—	dashboard() , productos()	KPIs + gráficos Recharts
AdminProductos.jsx	/admin/productos	Producto (+SKU), Categoría, Precio, Stock, Acciones	CRUD productos + categorias() + subirImagen()	Subida de imagen
AdminCategorías.jsx	/admin/categorias	ID, Nombre, Acciones	CRUD categorías	Modal size="sm" , un solo campo
AdminClientes.jsx	/admin/clientes	Cliente (+email), DNI, Contacto, Dirección, Acciones	CRUD clientes + buscarDni()	Autocompletado RENIEC
AdminPedidos.jsx	/admin/pedidos	Código, Cliente, Fecha, Ítems, Total, Estado, Acciones	CRUD pedidos + PagosAPI + comprobantes	Detalle con pago y boleta + cobrar
AdminVentas.jsx	/admin/ventas	Boleta, Pedido, Cliente(+DNI), Fecha, Op. gravada, IGV, Total, Pago, Estado, Acciones	comprobantes() , resumenVentas() , boletaUrl()	IGV 18% y filtros de fecha
AdminPagos.jsx	/admin/pagos	Código, Pedido, Cliente, Método, Monto, Estado, Referencia, Fecha, Acciones	PagosAPI.listar() , comprobanteUrl()	Fuera del menú
AdminUsuarios.jsx	/admin/usuarios	Usuario, Nombre, Email, Rol, Acciones	CRUD usuarios	Roles y contraseña opcional al editar

Login.jsx — el bloqueo con contador

Si el backend responde **429** (demasiados intentos), la respuesta trae `segundosRestantes` y la pantalla arranca una cuenta atrás:

```

if (err.status === 429 && err.data?.segundosRestantes) setBloqueo(err.data.segundosRestantes)

```

Mientras `bloqueo > 0` : los inputs se deshabilitan, el botón dice `Bloqueado (Ns)` y se muestra un `Alert` fijo en lugar del error normal. Un `useEffect` con `setInterval` baja el contador cada segundo, y otro detecta la transición a 0 (usando un `useRef` llamado `prevBloqueo`) para **limpiar el formulario** automáticamente. Además, si ya hay sesión guardada, redirige al panel sin mostrar el formulario.

`Dashboard.jsx` — KPIs y gráficos

Seis tarjetas KPI (`stat-card`) armadas desde un arreglo `stats` : Productos, Categorías, Clientes, Pedidos, Ventas (formateado con `money()`) y Stock bajo.

Dos gráficos de Recharts, ambos dentro de `<ResponsiveContainer>` :

- **BarChart** "Productos por categoría" — ⚠ los datos se agrupan **en el frontend** con un `reduce` sobre la lista de productos, no vienen del backend.
- **PieChart** (dona) "Estado del stock" — tres cortes calculados en el front: `> 10` , `1-10` y `0` , con los colores de `COLORES_STOCK = ['#10b981', '#f59e0b', '#ef4444']` .

Debajo: lista de "Stock bajo", ranking "Más vendidos" (ese **sí** viene calculado del backend, en `data.topProductos`) y "Acciones rápidas". Mientras carga muestra skeletons con la misma forma que el contenido final.

- 🖌 Colores del gráfico → `COLORES_STOCK` y el `fill="#6366f1"` del `<Bar>` .
- 🖌 Umbrales de stock → el bloque `estadoStock` .

`AdminProductos.jsx` — subida de imagen

El único CRUD con archivos. El input real está **oculto dentro de un** `<label>` estilizado como botón, un truco estándar para no usar el feo input de archivos del navegador:

```
<label className="btn btn-outline" style={{ cursor: 'pointer', margin: 0 }}>
  <i className="bi bi-upload" /> {subiendo ? 'Subiendo...' : 'Subir imagen'}
  <input type="file" accept="image/*" hidden onChange={subirImagen} disabled={subiendo} />
</label>
```

`subirImagen()` arma un `FormData` , llama a `AdminAPI.subirImagen(fd)` y guarda la URL devuelta en `form.imagen` , con vista previa inmediata. Se guarda en `uploads/productos` , máx. 5 MB.

Los badges de stock siguen la misma escala del dashboard: `> 10` verde, `1-10` ámbar, `0` rojo "Sin stock".

`AdminClientes.jsx` — RENIEC

El botón "Buscar" junto al DNI valida el formato y consulta RENIEC para autocompletar:

```
if (!/^\d{8}$/.test(form.dni)) { setFormError('Ingresa un DNI de 8 dígitos'); return }
const p = await AdminAPI.buscarDni(form.dni)
setForm((f) => ({ ...f, nombres: p.nombres, apellidos: p.apellidos })))
```

⚠ Al **editar**, el DNI queda bloqueado (`disabled={!editing}` , label "(no editable)") y el botón de RENIEC desaparece: el DNI es la identidad del cliente. Por eso `dupDni` solo se activa al crear (`activo: showModal && !editing`), mientras que `dupEmail` está activo siempre.

AdminPedidos.jsx — la página más completa

Tiene **cuatro modales** y varias funciones. Estados de pedido: `PENDIENTE`, `PAGADO`, `ENVIADO`, `ENTREGADO`, `CANCELADO`; métodos: solo `TARJETA` y `YAPE`. La función `badgeEstado()` decide el color del badge.

Modal	Qué hace
Nuevo pedido	Elige cliente y método, y arma los ítems uno a uno con un <code>draft</code> (<code>{productoId, cantidad}</code>) → <code>agregarItem()</code> / <code>quitarItem(id)</code> . El total se calcula en vivo con <code>reduce</code>
Editar estado	Solo cambia <code>estado</code> y <code>metodoPago</code>
Cobrar	Monta <code><PasarelaPago></code> ; al aprobarse, <code>onPagado</code> cierra y recarga
Ver detalle	★ Reúne tres cosas en una vista

El **detalle** (`verDetalle`) es la parte más interesante: pide en paralelo el listado de pagos y el de boletas, y cruza los datos para encontrar los de ese pedido:

```
const [pagos, boletas] = await Promise.all([PagosAPI.listar(), AdminAPI.comprobantes()])
const pago = pagos.filter((x) => x.pedidoId === p.id).sort((a, b) => b.id - a.id)[0] || null
const boleta = boletas.find((c) => c.pedidoId === p.id) || null
```

(El `sort` descendente por id se queda con el **pago más reciente**, por si hubo reintentos.) El modal `size="lg"` muestra: **Ítems** (ya venían dentro del pedido, no se piden aparte), **Pago** (código, método, estado, monto, referencia, últimos 4 dígitos, botón de comprobante) y **Boleta** (código, op. gravada, IGV, total, botón de descarga). Si no hay pago, ofrece un botón "Cobrar ahora" que cierra el detalle y abre la pasarela.

También hay un **reporte PDF** con filtros propios de fecha y estado (`repDesde`, `repHasta`, `repEstado`) que se descarga con `downloadBlob(AdminAPI.reportePedidosUrl({...}), 'reporte-pedidos.pdf')`.

Botones por fila, condicionales: ver detalle (siempre); cobrar (solo si no está `PAGADO` ni `CANCELADO`); ver boleta (solo si está `PAGADO`); editar y eliminar (siempre).

AdminVentas.jsx — registro de ventas con IGV

Lista las **boletas emitidas** (la base del libro de ventas). Arriba, cuatro tarjetas de resumen que vienen de `resumenVentas()`: Boletas emitidas, **Op. gravada** (subtotal), **IGV (18%)** y Total vendido.

⚠ Los montos y el IGV se calculan **en el backend**; el frontend solo los muestra con `money()`. Los filtros `desde` / `hasta` se aplican con el botón "Filtrar", que vuelve a llamar a `cargar()` con los mismos parámetros para la tabla y el resumen. Si se dejan vacíos, trae todo.

AdminPagos.jsx — la página fuera del menú

Su ruta existe en `App.jsx` pero **no aparece en el Sidebar**. Lo dice el propio comentario del código:

Pagos ya no está en el menú: el pago se ve dentro del detalle del pedido. La ruta queda para auditoría.

Es una tabla de solo lectura (sin crear/editar/eliminar) con la única acción de descargar el comprobante PDF. Se llega escribiendo `/admin/pagos` a mano. 🛠 Para reponerla en el menú, agrega su entrada al arreglo `links` de `Sidebar.jsx`.

7.3 Los componentes de `components/admin`

`AdminLayout.jsx` — arma `Sidebar` + (`Topbar` + `<Outlet />`) + `SessionTimer`. Maneja el sidebar colapsable y **recuerda la preferencia** en `localStorage` bajo `gs_sidebar` (`'1'` = colapsado). La clase `.collapsed` en el contenedor es la que dispara el CSS.

`Sidebar.jsx` — el menú se genera desde un arreglo, no está escrito a mano en el JSX:

```
const links = [
  { to: '/admin', label: 'Dashboard', icon: 'bi-grid-1x2-fill', end: true },
  { to: '/admin/productos', label: 'Productos', icon: 'bi-box-seam-fill' },
  // ... categorías, clientes, pedidos, ventas, usuarios
]
```

Usa `NavLink`, que aplica la clase `active` sola cuando la ruta coincide. La opción `end: true` del Dashboard evita que `/admin` se marque activo estando en `/admin/productos`. Abajo: "Volver a la tienda" y "Cerrar sesión".

 **Cómo agregar un ítem al menú:** añade un objeto a `links` con `to`, `label` e `icon` (nombre de Bootstrap Icons). Recuerda que además necesitas la ruta en `App.jsx` y la página en `pages/admin/`.

`Topbar.jsx` — título de la sección + chip del usuario (inicial del nombre, nombre y rol). El título sale de un mapa:

```
const TITLES = { '/admin': 'Dashboard', '/admin/productos': 'Productos', '/admin/categorias': 'Categorías',
  '/admin/clientes': 'Clientes', '/admin/pedidos': 'Pedidos' }
const title = TITLES[pathname] || 'Panel'
```

 **Ventas, Usuarios y Pagos no están en ese mapa**, así que muestran "Panel". Si agregas una página, agrégala también aquí.

`SessionTimer.jsx` — reloj flotante que cuenta cuánto le queda al access token. Decodifica el JWT **en el navegador** (sin librerías) leyendo el claim `exp`:

```
const payload = JSON.parse(atob(token.split('.')[1]))
return payload.exp ? payload.exp * 1000 : null
```

Un `setInterval` de 1 s **relee el token** cada vez (no guarda el tiempo aparte). Ese detalle es lo que hace que el contador se reinicie solo cuando `client.js` refresca el token: como el `exp` del token nuevo es mayor, el número sube. Si no hay token válido, el componente no se renderiza (`return null`). Con ≤ 60 s añade la clase `session-timer-low` (aviso visual).

-  El umbral de aviso → `const bajo = restante <= 60`.
-  El texto → `Sesión {m}:{String(s).padStart(2, '0')}`.
-  La **duración real** del token la decide el backend, no este componente.

`PasarelaPago.jsx` — modal de cobro con dos pestañas (`tab`: `'yape'` | `'tarjeta'`). Al montar pide `PagosAPI.config()` para traer número, titular, QR, tope y estado de Stripe.

Pestaña Flujo

Yape Muestra QR + número (con botón Copiar), pide el N° de operación (mínimo 6 dígitos) y permite **subir el voucher** reutilizando `AdminAPI.subirImagen()` . Luego `PagosAPI.pagarYape()`

Tarjeta Si `cfg.stripeEnabled` : `<Elements>` + `FormularioTarjetaStripe` . Si no: formulario propio con validación Luhn y `PagosAPI.pagarTarjeta()`

⚠ **Diferencia importante con el checkout público:** aquí el modal es de una sola vista, así que el botón "Pagar" **tokeniza y cobra en el mismo click** (no hay un paso intermedio donde el `CardElement` desaparezca).

Igual que en el checkout, `yapeBloqueado = pedido.total > (cfg?.montoMaximo || 500)` deshabilita la pestaña de Yape, y un `useEffect` cambia a "tarjeta" cuando llega la config si el monto la supera. Al terminar muestra `pago-ok` (con descarga de comprobante) o `pago-fail` (con botón Reintentar), y llama a `onPagado()` para que el padre recargue.

8. Estilos (`index.css`)

Un solo archivo, 751 líneas, sin frameworks CSS. No hay Tailwind ni Bootstrap (de Bootstrap solo se usan los **iconos**, cargados por CDN en `index.html`). Todo son clases escritas a mano.

8.1 Las variables de `:root`

Arriba del archivo hay un bloque de **tokens de diseño**. La regla del proyecto: los colores no se escriben sueltos en los componentes, se referencian con `var(--nombre)`. Por eso cambiar un token cambia toda la app de golpe.

Grupo	Variables	Para qué
Fondos	<code>--bg</code> , <code>--bg-2</code> , <code>--surface</code> , <code>--surface-2</code> , <code>--border</code> , <code>--border-strong</code>	Fondo de página, tarjetas y bordes
Texto	<code>--text-strong</code> , <code>--text</code> , <code>--muted</code> , <code>--faint</code>	Jerarquía tipográfica (escala slate)
Marca	<code>--accent</code> , <code>--accent-hover</code> , <code>--accent-soft</code> , <code>--accent-border</code> , <code>--accent-grad</code>	El color principal (indigo #4f46e5)
WhatsApp	<code>--whatsapp</code> , <code>--whatsapp-hover</code>	Botones verdes de contacto
Estados	<code>--success</code> , <code>--danger</code> , <code>--warning</code> (+ variantes <code>-soft</code> y <code>-text</code>)	Badges y alertas
Sombras	<code>--shadow-xs</code> , <code>--shadow-sm</code> , <code>--shadow</code> , <code>--shadow-lg</code>	Elevación en capas
Forma	<code>--r</code> (14px), <code>--r-sm</code> (10px), <code>--r-xs</code> (8px), <code>--t</code> (0.16s)	Redondeo y velocidad de transición

 **Cambiar el color de marca:** edita `--accent`, `--accent-hover`, `--accent-soft` y `--accent-grad` en `:root`. Botones, enlaces, badges, iconos del sidebar y gráficos cambian solos. (Ojo: los colores de los gráficos de Recharts están **hardcodeados** en `Dashboard.jsx`, hay que cambiarlos ahí aparte.)

8.2 Las familias de clases

El archivo está dividido en secciones con comentarios `/* ===== NOMBRE ===== */`. Estas son las familias que más vas a usar:

Familia	Clases principales	Dónde aparece
Botones	<code>btn</code> + <code>btn-primary</code> / <code>btn-outline</code> / <code>btn-ghost</code> / <code>btn-danger</code> / <code>btn-success</code> / <code>btn-whatsapp</code> , y tamaños <code>btn-sm</code> / <code>btn-lg</code> / <code>btn-block</code> / <code>btn-icon</code>	Toda la app
Formularios	<code>field</code> , <code>label</code> , <code>input</code> , <code>select</code> , <code>textarea</code> , <code>input-group</code> , <code>input-error</code> , <code>campo-error</code> , <code>form-grid</code> (+ <code>full</code> para ocupar dos columnas)	Modales y checkout
Badges	<code>badge</code> + <code>badge-ok</code> / <code>badge-warn</code> / <code>badge-danger</code> / <code>badge-accent</code> / <code>badge-cat</code> / <code>badge-yape</code> / <code>badge-tarjeta</code>	Estados en tablas
Tablas	<code>table-wrap</code> , <code>table-scroll</code> , <code>table</code> , <code>table-toolbar</code> , <code>table-search</code> , <code>table-count</code> , <code>sortable</code> , <code>is-sorted</code> , <code>sort-ind</code> , <code>cell-actions</code> , <code>thumb</code> , <code>pagination</code> , <code>page-btn</code>	Panel admin
Paneles	<code>panel</code> , <code>panel-head</code> , <code>panel-grid</code> , <code>page-head</code> , <code>grid-2</code>	Panel admin
Stat cards	<code>stat-grid</code> , <code>stat-card</code> , <code>stat-icon</code> , <code>stat-label</code> , <code>stat-value</code> , <code>chart-box</code> , <code>chart-legend</code> , <code>legend-dot</code>	Dashboard y Ventas
Modal	<code>modal-overlay</code> , <code>modal</code> , <code>modal-sm</code> , <code>modal-lg</code> , <code>modal-header</code> , <code>modal-body</code> , <code>modal-footer</code> , <code>modal-close</code>	Todos los modales
Toast	<code>toaster</code> , <code>toast</code> , <code>toast-success</code> / <code>toast-error</code> / <code>toast-info</code> , <code>toast-icon</code> , <code>toast-msg</code> , <code>toast-close</code>	Notificaciones
Tienda	<code>hero</code> , <code>section</code> , <code>section-title</code> , <code>eyebrow</code> , <code>feature-grid</code> , <code>product-grid</code> (+ <code>grid-3</code> / <code>grid-4</code>), <code>product-card</code> , <code>catalog-layout</code> , <code>filter-card</code> , <code>filter-link</code> , <code>detail-grid</code> , <code>spec-card</code> , <code>footer-grid</code>	Páginas públicas
Carrito	<code>carrito-list</code> , <code>carrito-item</code> , <code>carrito-item-qty</code> , <code>carrito-resumen</code> , <code>carrito-total</code> , <code>cart-badge</code>	Navbar y carrito
Checkout	<code>stepper</code> , <code>step</code> , <code>step-circle</code> , <code>step-line</code> , <code>wizard-nav</code> , <code>opcion-card</code> (+ <code>sel</code> , <code>opcion-card-off</code>), <code>checkout-grid</code> , <code>checkout-resumen</code> , <code>revision-bloque</code> , <code>procesando-pasos</code> , <code>compra-ok</code>	Checkout
Pasarela	<code>pasarela-tabs</code> , <code>pasarela-tab</code> , <code>pasarela-yape</code> , <code>pasarela-qr</code> , <code>pasarela-cuenta</code> , <code>pasarela-guia</code> , <code>stripe-card</code> (+ <code>focus</code> , <code>err</code>), <code>marca-badge</code> , <code>aviso-demo</code> , <code>pago-ok</code> , <code>pago-fail</code>	Checkout y PasarelaPago
Carga	<code>skeleton</code> , <code>sk-card</code> , <code>sk-row</code> , <code>sk-line</code> , <code>spinner</code> , <code>spinner-sm</code> , <code>empty</code>	Estados de carga y vacío
Layout admin	<code>admin</code> , <code>collapsed</code> , <code>sidebar</code> , <code>sidebar-link</code> , <code>sidebar-brand</code> , <code>admin-main</code> , <code>admin-content</code> , <code>topbar</code> , <code>userchip</code> , <code>session-timer</code>	Panel
Login	<code>login-wrap</code> , <code>login-card</code> , <code>login-side</code> , <code>login-form</code>	Login

Al final del archivo hay una sección **RESPONSIVE** con los `@media` que colapsan el sidebar, convierten el menú en hamburguesa (`.nav-toggle` / `.nav-links.open`) y pasan las grillas a una columna.

 **Dónde agregar clases nuevas:** al final de la sección temática que corresponda (por ejemplo, un botón nuevo va junto a los demás `btn-*`). Usa siempre `var(--...)` en vez de códigos de color literales, y respeta la convención de nombres en español que ya existe (`pasarela-` , `carrito-` , `detalle-`).

⚠ Si cambias un nombre de clase existente, búscalo antes en `src/` : al no haber CSS Modules, nada avisa si rompes un estilo.

9. Guía rápida: "quiero cambiar X, ¿dónde toco?"

#	Quiero...	Archivo y punto exacto
1	Cambiar el color principal de la marca	<code>index.css</code> → <code>:root</code> → <code>--accent</code> , <code>--accent-hover</code> , <code>--accent-soft</code> , <code>--accent-grad</code>
2	Cambiar el logo o el nombre de la tienda	<code>components/public/Navbar.jsx</code> (<code>.nav-brand</code>) y <code>components/admin/Sidebar.jsx</code> (<code>.sidebar-brand</code>)
3	Agregar una página al panel	3 pasos: crear el archivo en <code>pages/admin/</code> → agregar <code><Route></code> en <code>App.jsx</code> → agregar el objeto al arreglo <code>links</code> de <code>Sidebar.jsx</code> (y opcionalmente a <code>TITLES</code> de <code>Topbar.jsx</code>)
4	Agregar un endpoint	<code>api/endpoints.js</code> , en el grupo que corresponda (<code>AuthAPI</code> / <code>PublicAPI</code> / <code>AdminAPI</code> / <code>PagosAPI</code>)
5	Cambiar el tiempo del toast (3.5 s)	<code>components/ui/Toast.jsx</code> → <code>setTimeout(() => quitar(id), 3500)</code>
6	Cambiar cuánto tarda en borrarse un error (5 s)	<code>hooks/useAutoClear.js</code> → parámetro <code>ms = 5000</code> , o el 3.er argumento en la llamada
7	Cambiar el debounce del duplicado (500 ms)	<code>hooks/useDuplicado.js</code> → <code>delay = 500</code> ; el mínimo de caracteres, <code>minLargo = 3</code>
8	Cambiar las filas por página (8)	El <code>pageSize</code> que pasa cada página admin a <code>useTableControls</code> , o el default del hook
9	Cambiar por qué columnas busca una tabla	El <code>searchKeys</code> de esa página (ej. <code>['nombre', 'categoriaNombre']</code>) en <code>AdminProductos.jsx</code>)
10	Cambiar los textos de la tienda	La página: <code>pages/public/Home.jsx</code> (hero y features), <code>Contacto.jsx</code> (arreglos <code>info</code> y <code>pasos</code>), <code>components/public/Footer.jsx</code>
11	Cambiar los pasos del checkout	<code>pages/public/Checkout.jsx</code> → <code>PASOS_LABEL</code> + los bloques <code>{step === N && ...}</code> 
12	Cambiar los avisos de demo del pago	<code>Checkout.jsx</code> y <code>components/admin/PasarelaPago.jsx</code> → bloques <code>.aviso-demo</code>
13	Cambiar el contador de sesión	<code>components/admin/SessionTimer.jsx</code> → umbral <code>restante <= 60</code> y el texto.  La duración real la fija el backend
14	Cambiar el puerto o el proxy	<code>frontend/vite.config.js</code> → <code>server.port</code> y el bloque <code>proxy</code>
15	Cambiar los estados de un pedido	<code>pages/admin/AdminPedidos.jsx</code> → <code>const ESTADOS = [...]</code> ( deben coincidir con el backend) y <code>badgeEstado()</code> para los colores
16	Cambiar los colores de los gráficos	<code>pages/admin/Dashboard.jsx</code> → <code>COLORES_STOCK</code> y el <code>fill</code> del <code><Bar></code>
17	Cambiar los umbrales de stock (verde/ámbar/rojo)	<code>Dashboard.jsx</code> (<code>estadoStock</code>) y <code>AdminProductos.jsx</code> (los ternarios <code>p.stock > 10 ? ... : p.stock > 0 ? ...</code>)

#	Quiero...	Archivo y punto exacto
1 8	Cambiar el tope de Yape (S/ 500)	Backend (<code>/api/config</code>); el respaldo local está en <code>config/ConfigContext.jsx</code> → <code>DEFAULTS.yapeMontoMaximo</code>
1 9	Cambiar cuántos destacados muestra el inicio	<code>pages/public/Home.jsx</code> → <code>list.slice(0, 8)</code>
2 0	Cambiar el formato del precio o el SKU	<code>utils/format.js</code> → <code>money()</code> y <code>sku()</code>
2 1	Cambiar los roles de usuario	<code>pages/admin/AdminUsuarios.jsx</code> → <code>const ROLES = ['ADMIN', 'USUARIO']</code> (⚠ debe coincidir con el backend)
2 2	Volver a poner Pagos en el menú	<code>components/admin/Sidebar.jsx</code> → agregar <code>{ to: '/admin/pagos', label: 'Pagos', icon: 'bi-credit-card' }</code> a <code>links</code>
2 3	Cambiar el título de la pestaña o el favicon	<code>frontend/index.html</code> → <code><title></code> y <code><link rel="icon"></code>
2 4	Cambiar la tipografía	<code>frontend/index.html</code> (el <code><link></code> de Google Fonts) + <code>index.css</code> → <code>body { font-family: ... }</code>
2 5	Cambiar dónde se compila el build	<code>vite.config.js</code> → <code>build.outDir</code> ⚠ (Spring espera la SPA en <code>src/main/resources/static</code>)

Cómo ejecutarlo

```
cd frontend
npm install
npm run dev      # http://localhost:5173 (necesita el backend en :8080)
npm run build    # compila hacia ../src/main/resources/static
npm run preview  # sirve el build compilado
```